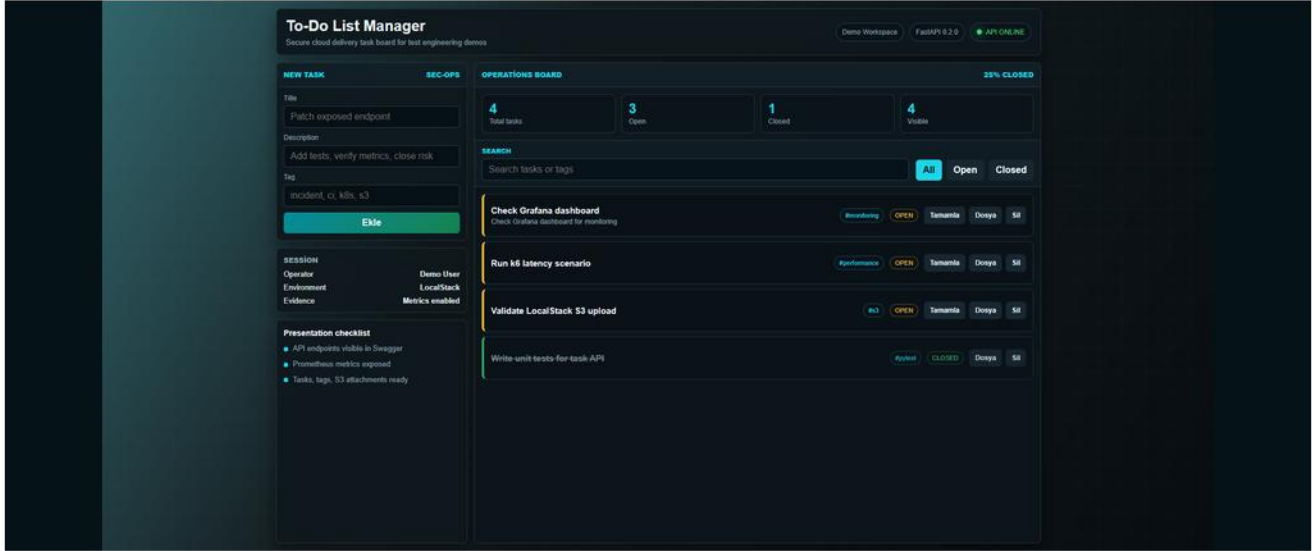


1. Giriş ve Proje Amacı

Bu proje, To-Do List Manager konusunu temel alarak küçük bir FastAPI mikroservisi üzerinde test mühendisliği ve bulut dağıtım hattının uçtan uca gösterilmesini amaçlar. Odak; karmaşık bir uygulama geliştirmek değil, basit bir servis için ölçülebilir kalite, otomasyon ve gözlemlenebilirlik altyapısı kurmaktır.



Şekil 1. Uygulamanın tek ekran görev yönetimi arayüzü.

20 test

Unit, integration ve E2E testleri başarıyla geçmiştir.

%89 coverage

Şartnamedeki minimum %70 coverage hedefinin üzerindedir.

31.44 ms p95

k6 performans testinde 500 ms eşliğinin altında kalmıştır.

Kapsam

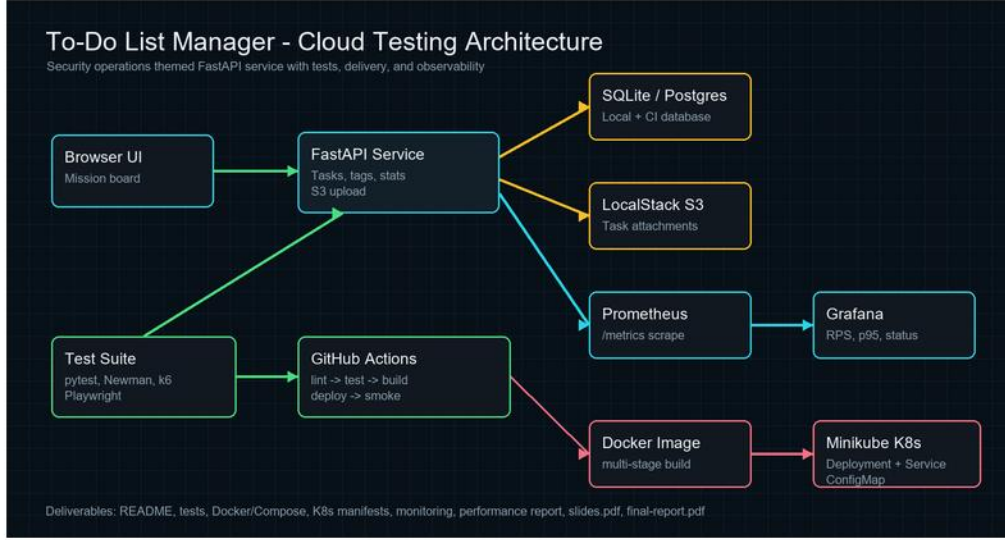
Uygulama Task ve Tag entity'leri ile görev oluşturma, listeleme, tekil okuma, tamamlama, silme, etiketleme, dosya yükleme ve istatistik uç noktalarını sağlar. Teslim kapsamı README, Dockerfile, docker-compose.yml, Kubernetes manifestleri, monitoring dosyaları, Postman koleksiyonu, k6 senaryosu, testler, mimari diyagram, slaytlar ve final raporundan oluşur.

Proje Yorumu

Arayüz görevleri operasyon paneli mantığıyla sunar. Böylece test, dağıtım ve izleme çıktıları yalnızca teknik dosyalar olarak değil, kullanıcı tarafından anlaşılabilen bir iş akışı olarak da gösterilir.

2. Mimari Tasarım

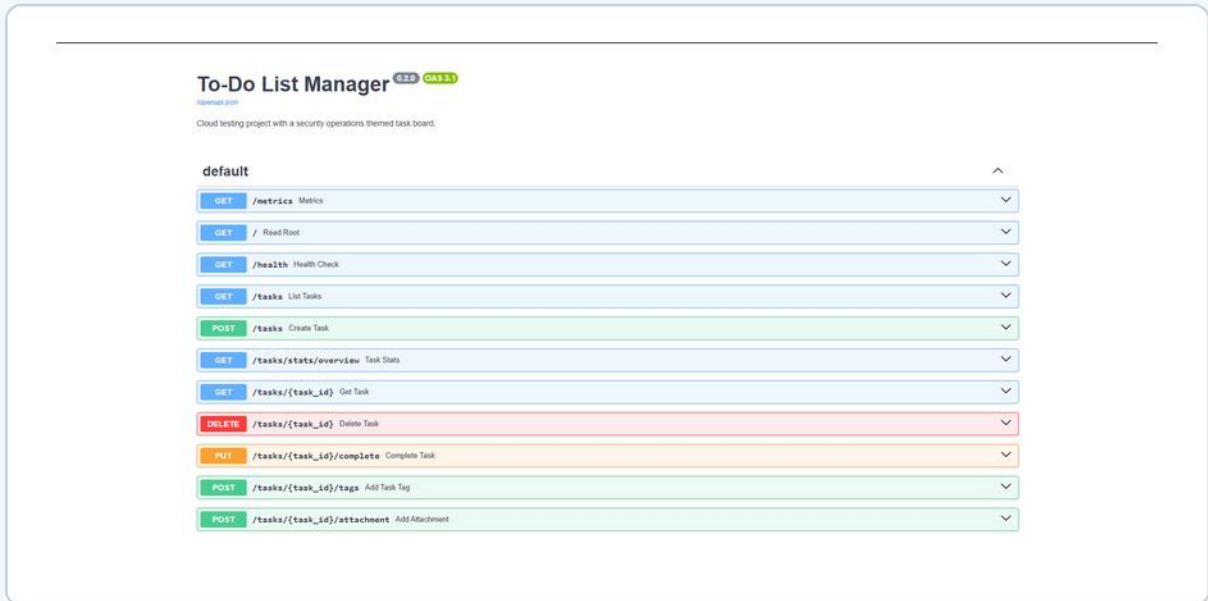
Sistem FastAPI tabanlı web/API servisi, yerel geliştirme için SQLite, Docker ve CI ortamı için PostgreSQL, dosya yükleme için LocalStack S3, metrik toplama için Prometheus ve görselleştirme için Grafana bileşenlerinden oluşur. Docker Compose yerel orkestrasyon sağlar; Kubernetes tarafında Deployment, Service ve ConfigMap manifestleri Minikube üzerinde doğrulanmıştır.



Şekil 2: To-Do List Manager mimari görünümü.

API Sözleşmesi

REST API; sağlık kontrolü, görev listeleme, görev oluşturma, tekil görev okuma, görev tamamlama, görev silme, etiket ekleme, dosya yükleme ve istatistik uç noktalarını içerir. Swagger/OpenAPI ekranı API sözleşmesini görünür kılar.



Şekil 3: Swagger/OpenAPI dokümantasyonu.

3. Test Stratejisi

Test yaklaşımı şartnamedeki test mühendisliği katmanlarını kapsayacak şekilde tasarlanmıştır. Unit testler Task API davranışını ve S3 servis mantığını doğrular. Integration testler PostgreSQL Testcontainers altyapısıyla veritabanı davranışını sınar. Playwright E2E testleri kullanıcı arayüzünün açılması, görev ekleme ve temel iş akışları için koruma sağlar. Postman koleksiyonu Newman ile smoke test olarak CI hattında çalıştırılır.

```
(base) PS C:\Users\mdapc\OneDrive\Masaüstü\bulutproje\bulut-test-project> pytest --cov=src --cov-report=term-missing
===== test session starts =====
platform win32 -- Python 3.11.15, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\mdapc\OneDrive\Masaüstü\bulutproje\bulut-test-project
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.12.1, Faker-40.19.1, base-url-2.1.0, cov-7.1.0, playwright-0.8.0
collected 20 items

tests\e2e\test_ui.py ... [ 15%]
tests\integration\test_db.py ... [ 30%]
tests\unit\test_s3.py ..... [ 50%]
tests\unit\test_tasks.py ..... [100%]

===== warnings summary =====
..\..\..\miniconda3\Lib\site-packages\fastapi\testclient.py:1
C:\Users\mdapc\miniconda3\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using 'httpx' with 'starlette.testclient' is deprecated; install
'httpx2' instead.
  from starlette.testclient import TestClient as TestClient # noqa

tests\e2e\test_ui.py::test_homepage_loads[chromium]
C:\Users\mdapc\miniconda3\Lib\site-packages\websockets\legacy\__init__.py:6: DeprecationWarning: websockets.legacy is deprecated; see https://websockets.readthedoc
s.io/en/stable/howto/upgrade.html for upgrade instructions
  warnings.warn( # deprecated in 14.0 - 2024-11-09

tests\e2e\test_ui.py::test_homepage_loads[chromium]
C:\Users\mdapc\miniconda3\Lib\site-packages\uvicorn\protocols\websockets\websockets_impl.py:17: DeprecationWarning: websockets.server.WebSocketServerProtocol is de
precated
  from websockets.server import WebSocketServerProtocol

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== tests coverage =====
coverage: platform win32, python 3.11.15-final-0

Name                Stmts   Miss  Cover   Missing
-----
src\__init__.py       0       0   100%
src\main.py          102     16    84%   68-69, 145, 160-178
src\models.py         17       0   100%
src\services\__init__.py  0       0   100%
src\services\s3.py    22       0   100%
TOTAL                 141     16    89%

===== 20 passed, 3 warnings in 8.65s =====
```

Şekil 4. Pytest coverage sonucu: 20 test geçti, toplam coverage %89.

Unit

Task API ve S3 servis mantığı.

Integration

PostgreSQL Testcontainers ile DB doğrulama.

E2E

Playwright ile UI senaryoları.

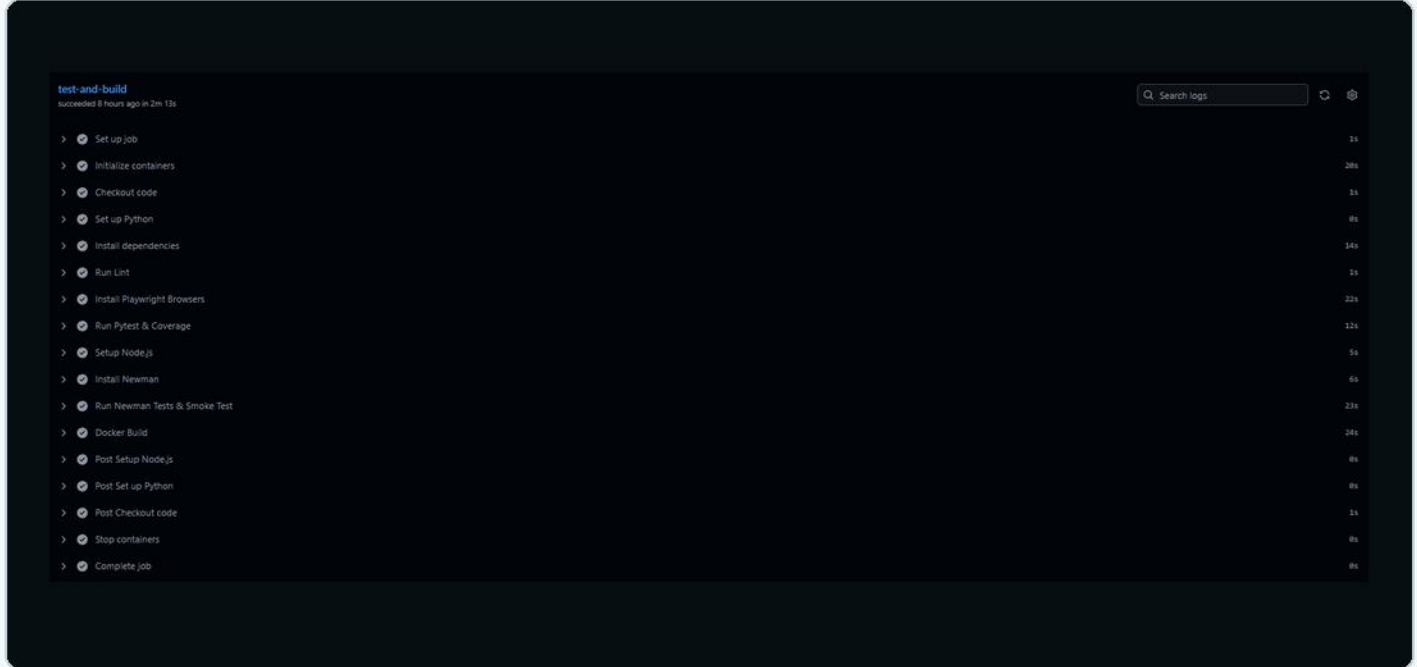
Coverage

Toplam %89, hedefin üzerinde.

Coverage hedefi şartnamede belirtilen minimum %70 seviyesinin üzerindedir. Uyarılar üçüncü parti paketlerin deprecation uyarılarıdır ve test başarısını etkilememektedir.

4. CI/CD ve Dağıtım

GitHub Actions workflow'u push ve pull request durumlarında kalite kapılarını otomatik çalıştırır. Hatta checkout, Python kurulumu, bağımlılık kurulumu, Black lint kontrolü, Playwright Chromium kurulumu, pytest coverage, Node.js/Newman smoke test ve Docker build adımları bulunur.



Şekil 5. Başarılı GitHub Actions run detay ekranı.

Yerel dağıtım Docker Compose ile API, PostgreSQL, LocalStack, Prometheus ve Grafana servislerini ayağa kaldırır. Kubernetes dağıtımı Minikube üzerinde doğrulanmıştır; podların Running durumda görünmesi uygulamanın cluster ortamında çalıştığını gösterir.

```
(base) PS C:\Users\mdapc\OneDrive\Masaüstü\bulutproje\bulut-test-project> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
todo-list-manager-67d8c764f7-lp9rv  1/1     Running   0           28m
todo-list-manager-67d8c764f7-txtmk  1/1     Running   0           28m
(base) PS C:\Users\mdapc\OneDrive\Masaüstü\bulutproje\bulut-test-project> kubectl get svc
NAME                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kubernetes          ClusterIP   10.96.0.1    <none>         443/TCP           29m
todo-list-manager-service NodePort    10.99.192.88 <none>         80:30080/TCP     29m
```

Şekil 6. Minikube üzerinde podların Running durumda olması.

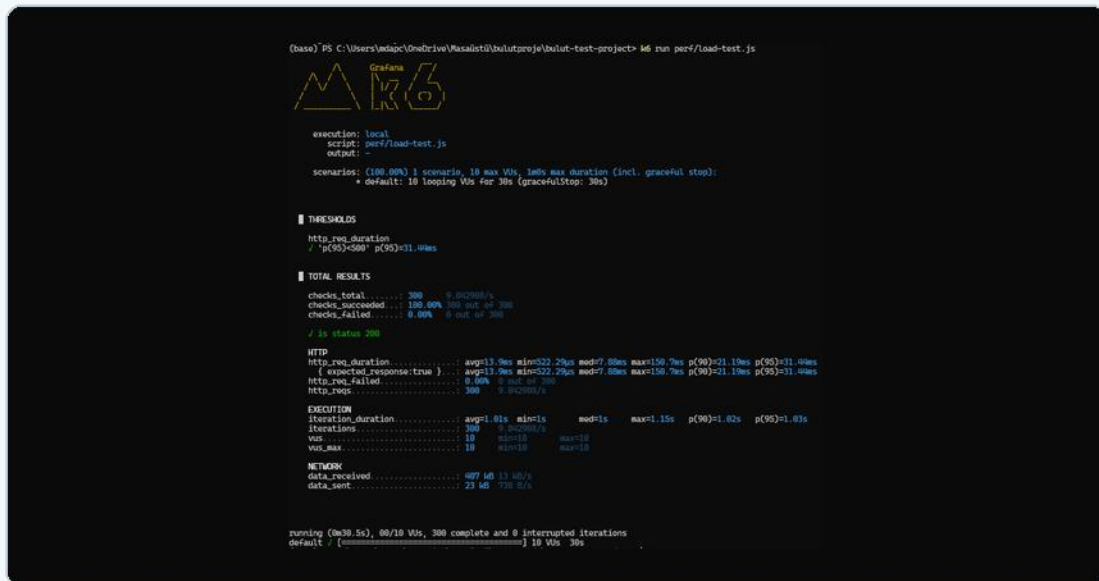
5. Performans ve Gözlemlenebilirlik

Prometheus /metrics endpoint'i uygulama istekleri, HTTP durumları ve yanıt süreleri için metrik üretir. Grafana dashboard'u request rate, P95 latency ve HTTP status code panelleri ile servis davranışını görselleştirir.



Şekil 7. Grafana dashboard: request rate, P95 latency ve HTTP status panelleri.

k6 performans testi 10 sanal kullanıcıyla 30 saniyelik senaryoyu çalıştırmıştır. Sonuçlarda HTTP başarısız istek oranı %0, P95 latency 31.44 ms ve threshold sonucu başarılıdır.



Şekil 8. k6 performans testi sonucu.

6. Sonuç ve Değerlendirme

Proje, şartnamede beklenen ana bileşenleri küçük ama savunulabilir bir servis üzerinden tamamlar: REST API, veritabanı, LocalStack S3 entegrasyonu, unit/integration/E2E testler, Postman/Newman smoke test, Docker, Kubernetes, GitHub Actions, Prometheus, Grafana ve k6 performans testi.

Metrik	Sonuç
Test sayısı	20 passed
Coverage	%89
Performans	p95 31.44 ms
CI/CD	Başarılı run
Kubernetes	Podlar Running

Genel Yorum

Çalışma, küçük bir mikroservis için kalite sinyallerinin yalnızca kod seviyesinde değil, teslimat ve izleme süreçlerinde de görünür hâle getirilebileceğini gösterir. Bu nedenle proje hem teknik gereksinimleri karşılar hem de sunum sırasında canlı demo ile savunulabilecek somut kanıtlar üretir.

Kaynakça

FastAPI, SQLAlchemy, pytest, Testcontainers, LocalStack, Docker, Kubernetes, GitHub Actions, Prometheus, Grafana, k6 ve Playwright dokümantasyonları proje geliştirme sürecinde referans alınmıştır.